CLIENT DEMO PREPARATION

Engineering audit

StockPilot AI

Bugs, data integrity risks and demo readiness

Prepared for the project team and colleagues. Review date: **06 November 2026** (Asia/Kolkata). Source revision: **c2e7496**.

Target demo: **web app + real backend + Tally/Excel imports**. Mobile and frontend-only demo mode are included for completeness and are marked by their conditions.

41 findings	20 P1 items	21 P2 items
6 review areas	265 backend tests passed	47 web tests passed

Recommended decision

Stabilize the demonstrated workflows before the client rehearsal. Prioritize overlapping-file deduplication, Excel dates, imported-sales analytics, database integrity and reliable screen updates. A passing test suite does not establish correct behavior in the uncovered cases below.

This is an audit and proposed repair backlog. No application fixes, migrations, deployments or business-data changes were performed. It identifies the issues found in this review; it is not a guarantee that every possible defect has been discovered.

Read first

IM-01 / IM-02: duplicate vouchers and Excel dates.
BE-01 / BE-02: forecasts and profit consistency.
DB-05 / DB-06 / DB-10: reset dependencies, persisted precision and mutable base units.
FE-02 / FE-04 / FE-05: workspace context, stale values and hidden records.

P1 means repair or exclude the affected workflow before demonstration/use. Conditional P1 items, such as frontend-only demo mode or hosted storage, apply only when that configuration is used.

HOW TO INTERPRET THIS REPORT

Evidence and coverage

Evidence label	Meaning
Reproduced	Actual project methods or client source executed with isolated in-memory inputs/mocks; not a live client-data or UI/database reproduction.
Code-confirmed	The behavior follows from current source, schema or configuration. Runtime, concurrency or failure-injection verification remains to be performed where stated.
Configuration risk	The supplied deployment configuration has an unresolved operational risk; live infrastructure was not inspected.

Verification already completed

Check	Result and limits
Backend suite	275 executions: 265 passed, 10 skipped. Tests used isolated H2 fixtures. JDK 25 required an explicit Byte Buddy agent and compatibility flag. Java 21 remains the project target.
Web checks	47 tests passed and lint passed. Production build passed with --webpack. Default Turbopack hit a sandbox port-binding restriction; this is not classified as an app defect.
Mobile check	TypeScript typecheck passed. No physical-device or emulator walkthrough was performed.
Targeted probes	Verified overlap duplication, Excel dates, CSV escaping, party GSTIN validation, raw-row loss, demo-mode behavior, cross-tab sessions, decimal input validity and local-date defaults.

Open verification gates

No live client database was connected or modified. PostgreSQL-specific migration checks and crash/replay scenarios were not run in this review; 10 backend checks were skipped. Hosted configuration, backups/restoration, large-file performance and full browser/device behavior remain unverified.

The test totals come from the current review session at the same source revision. The expanded database findings were not converted into new regression tests because application changes have not yet been authorized.

Scope included web routes and API clients; backend services, authorization and DTOs; all migration files and relevant persistence paths; Classic/Smart Import; mobile screens; and local/deployment configuration. This is not a penetration test or dependency vulnerability audit.

USE STABLE IDS IN TEAM DISCUSSIONS

Finding map

Area	Items	Where to start	Page
Frontend	12	Session context, transactions, pagination and display accuracy	4
Database and persistence	10	Constraints, transactions, rounding and lifecycle dependencies	8
Tally, Excel and CSV imports	5	Duplicate protection, dates, CSV and staging validation	12
Backend, analytics and permissions	6	Demand, cost/profit, permissions and collections	14
Mobile workflows	5	Duplicate submissions, sessions, availability and navigation	16
Setup and operational reliability	3	Reproducible startup, hosted storage and stalled requests	18
Repair and rehearsal plan	-	Dependency-aware batches and acceptance gates	19
Limits and retained safeguards	-	Intentional feature limits and gaps still to verify	20

Severity

P1 - protect correctness or access: incorrect or duplicated business records, misleading core intelligence, sensitive-field exposure, blocked important workflows or conditional data-loss risk.

P2 - repair before the affected screen is shown: usability failures, misleading display states, incomplete navigation, precision presentation, validation or reliability problems.

Read each finding as one repair workstream. Some workstreams share a cause: DB-03 covers payment atomicity while BE-06 covers partial-payment rules; DB-06 covers stored precision while BE-02 covers unit math. They should be tested together, not counted as independent product features.

Source references

Paths are relative to the repository root; numbers identify the inspected source line at revision c2e7496. References point to current implementation or original constraints. Any database repair must use new Flyway migrations rather than editing merged migrations.

No credentials, client records, source-export contents or local-machine absolute paths are included. The only external reference is official Render storage documentation supporting OP-02.

12 FINDINGS / FE

Frontend

Web UI behavior and the API contract. FE-01 applies only to frontend-only demo mode; your selected real-backend demo should keep that mode disabled. FE-11 applies when a non-INR currency is configured.

FE-01 P1 Demo-mode transactions contradict entered values

Evidence: Reproduced - isolated probe

Trigger: Enable frontend demo mode, then adjust stock, create a sale or purchase, or place a dealer order. **Impact:** Stock remains unchanged, invoice totals are fixed at unrelated amounts, and newly placed dealer orders disappear from subsequent reads.

Fix: Use prepared backend data for the demo or implement consistent simulated transactions, ledger updates, calculated totals, and persisted orders.

Verify: Create a known quantity/rate transaction, assert its exact total and stock effect, then reload its invoice and dealer order.

Sources: web/lib/demoApi.ts:642
web/lib/demoApi.ts:647
web/lib/demoApi.ts:857

FE-02 P1 Cross-tab account changes leave stale workspace context

Evidence: Reproduced - isolated probe

Trigger: Keep workspace A open in one tab, sign into workspace B in another, then submit a new product from the original tab. **Impact:** The original tab retains workspace A cache and context while its request uses workspace B authorization, risking an unintended workspace write.

Fix: Synchronize identity changes through storage events or BroadcastChannel, clear cached and unsaved workspace state, and scope query keys by session.

Verify: Use two tabs with different accounts; switching either session must clear or reload the other before another business write is possible.

Sources: web/lib/api.ts:88
web/lib/api.ts:124
web/lib/query.tsx:17

FE-03 P2 Saved invoices retain the previous invoice number

Evidence: Code-confirmed

Trigger: Create a sale or purchase successfully, change transaction details, then submit another transaction without manually replacing the invoice number. **Impact:** The backend rejects the second transaction because the completed form retains its existing invoice number and does not clearly transition to another transaction.

Fix: Reset completed transaction inputs, generate a fresh invoice number, and show a confirmation containing the saved invoice reference.

Verify: Create two consecutive sales and purchases from their respective pages; each must receive a distinct invoice number and visible success confirmation.

Sources: web/app/sales/page.tsx:32
web/app/purchases/page.tsx:31
backend/src/main/java/com/stockpilot/ai/service/SalesService.java:59

FE-04 P2 Writes leave related stock and balances stale

Evidence: Code-confirmed

Trigger: Create a sale, purchase, or legacy import, then immediately inspect related product stock, party balances, or dashboard values. **Impact:** Partial query invalidation leaves related screens or active selectors displaying pre-transaction values, undermining consistency during a live demonstration.

Fix: Centralize invalidation of all affected query families after successful writes, including products, parties, ledger views, and derived dashboard metrics.

Verify: For each write workflow, compare affected screens with direct API reads immediately afterward without a manual refresh.

Sources: web/app/sales/page.tsx:32
web/app/purchases/page.tsx:31
web/app/imports/page.tsx:175

FE-05 P2 First-page limits hide records and understate totals

Evidence: Code-confirmed

Trigger: Import more than 100 customers or 200 products, or create more invoices than the pages' fixed requested sizes. **Impact:** Records beyond the first page are inaccessible through affected lists or selectors, and customer total receivables excludes customers outside the loaded page.

Fix: Add pagination and server-side searchable selectors; retrieve aggregate receivables separately from paginated customer rows.

Verify: Load datasets exceeding each page limit, select a later product, navigate later invoices, and reconcile displayed receivables with the backend aggregate.

Sources: web/app/sales/page.tsx:17
web/app/customers/page.tsx:47
web/app/customers/page.tsx:130

FE-06 P2 Failed actions and requests lack visible feedback

Evidence: Code-confirmed

Trigger: Cause an API failure while using AI chat, dealer ordering, forecasting, data cleanup, warehouse creation, or affected list queries. **Impact:** Actions appear unresponsive and failed reads can resemble empty data or zero balances, leaving users unable to diagnose or recover.

Fix: Render request-specific loading, error, retry, and success states; disable concurrent AI prompts and route downloads through shared session-expiry handling.

Verify: Inject representative validation, permission, server, network, and expired-session failures; ensure every affected workflow provides an actionable visible response.

Sources: web/app/assistant/page.tsx:29
web/app/portal/page.tsx:28
web/app/forecasts/page.tsx:18

FE-07 P2 Page actions ignore role permissions

Evidence: Code-confirmed

Trigger: Sign in as Viewer or Manager and open product, stock, forecast, cleanup, or report pages containing unauthorized write/export controls. **Impact:** Users encounter controls the backend rejects, causing avoidable demo failures; hidden mutation errors can make these buttons appear broken.

Fix: Use shared permission checks for actionable page content and direct routes while preserving authoritative backend authorization.

Verify: Exercise each advertised role and confirm only permitted actions are available; direct forbidden API requests must still be rejected.

Sources: web/app/products/page.tsx:86
web/app/stock/page.tsx:60
backend/src/main/java/com/stockpilot/ai/service/PermissionService.java:201

FE-08 P2 Decimal payments fail and amounts lose paise

Evidence: Reproduced - isolated probe

Trigger: Enter a payment or reminder amount such as 100.50, or view prices and invoice totals containing paise. **Impact:** Native input validation rejects decimal payment amounts, while the shared monetary formatter rounds transaction values to whole rupees.

Fix: Use decimal input steps and suitable positive minimums; retain two decimals for transactional values and explicitly label any rounded summaries.

Verify: Submit a 100.50 payment and verify it reduces outstanding by exactly 100.50; confirm matching two-decimal displays across transactional screens.

Sources: web/app/customers/page.tsx:198
web/app/customers/page.tsx:212
web/lib/api.ts:150

FE-09 P2 Dealer cart cannot correct accidental additions

Evidence: Code-confirmed

Trigger: Add an unwanted dealer product or click Add one extra time before placing the order. **Impact:** The cart supports increments only, so the customer cannot decrease or remove items without abandoning or reloading the page.

Fix: Provide editable quantities, remove-item and clear-cart controls, with positive quantity validation and clear order feedback.

Verify: Add multiple products, decrease a quantity, remove a product, clear the cart, and confirm the submitted order matches the final selection.

Sources: web/app/portal/page.tsx:41
web/app/portal/page.tsx:70
web/app/portal/page.tsx:82

FE-10 P2 Default transaction dates use the wrong local day

Evidence: Reproduced - isolated probe

Trigger: Open sales, purchases, or customer payment/reminder forms before 05:30 India time, when the UTC calendar date is still yesterday. **Impact:** Invoice and payment dates default to yesterday; a reminder intended for tomorrow defaults to today, potentially misdating business records.

Fix: Use a shared local-calendar or workspace-timezone date helper instead of slicing UTC ISO timestamps.

Verify: Freeze the clock at 00:15 India time and assert today's transaction date and tomorrow's reminder date, including month and year boundaries.

Sources: web/app/sales/page.tsx:16
web/app/purchases/page.tsx:15
web/app/customers/page.tsx:26

FE-11 P2 Configured currency is ignored by monetary displays

Evidence: Code-confirmed

Trigger: Configure a supported-looking non-INR currency through onboarding or settings and inspect monetary values throughout the application. **Impact:** The backend stores the chosen currency but frontend formatting still labels amounts as INR or rupees, misrepresenting monetary units.

Fix: Use the workspace currency throughout formatting or explicitly restrict configuration to INR until other currencies are supported.

Verify: Configure an alternative currency and inspect dashboard, invoices, catalog, supplier balances, and portal totals for consistent monetary units.

Sources: web/app/settings/page.tsx:126
web/lib/api.ts:152
backend/src/main/java/com/stockpilot/ai/service/TenantService.java:37

FE-12 P2 Sales availability ignores the selected warehouse

Evidence: Code-confirmed

Trigger: Select a warehouse with no available units while the same product has positive stock in another warehouse. **Impact:** The product selector displays total stock across warehouses, but submission fails the selected-warehouse check, contradicting the apparent availability.

Fix: Display and validate availability for the selected warehouse, refreshing it when product or warehouse selection changes.

Verify: Create different quantities in two warehouses and confirm the sales form shows each selected warehouse's availability and handles insufficient stock clearly.

Sources: web/app/sales/page.tsx:67
backend/src/main/java/com/stockpilot/ai/service/CatalogService.java:164
backend/src/main/java/com/stockpilot/ai/service/SalesService.java:80

10 FINDINGS / DB

Database and persistence

Data constraints and persistence behavior. These findings are based on source/schema inspection, not a new migrated PostgreSQL reproduction. Prioritize isolated PostgreSQL regression coverage before applying migrations or repairing historical data.

DB-01 P1 Supplier invoice identity is too broad

Evidence: Code-confirmed; new database reproduction not run

Trigger: Two different suppliers submit their own invoice numbered 001, or document numbers are reused across financial years. **Impact:** Tenant-wide invoice-number uniqueness rejects legitimate purchases and can block Smart Import.

Fix: Define supplier, financial-year and source-voucher identity; update checks and constraints together with an append-only migration.

Verify: Different suppliers can reuse numbers while the same source invoice remains idempotent across imports.

Sources: backend/src/main/java/com/stockpilot/ai/service/PurchaseService.java:59
backend/src/main/resources/db/migration/V1__init_schema.sql:297
backend/src/main/java/com/stockpilot/ai/service/SmartImportStagingService.java:1783

DB-02 P1 Product merges do not preserve unit-dependent history

Evidence: Code-confirmed; database reproduction not run

Trigger: Merge products whose base units or product-specific pack conversions differ. **Impact:** Raw product-ID reassignment can reinterpret stock quantities and leave historical invoices without the conversions needed for profit calculations.

Fix: Reject incompatible merges and reconcile unit conversions and dependent records before moving history.

Verify: Merge compatible fixtures and reject KG-to-PCS merges; quantities, valuations and historical profits remain unchanged.

Sources: backend/src/main/java/com/stockpilot/ai/service/DataQualityService.java:129
backend/src/main/java/com/stockpilot/ai/repo/Repositories.java:94
backend/src/main/java/com/stockpilot/ai/service/DashboardService.java:198

DB-03 P1 Customer payment posting is not atomic or retry-safe

Evidence: Code-confirmed; failure and replay tests not run

Trigger: A failure occurs after saving a payment, or a client resubmits a payment whose response was lost. **Impact:** Payment, reminder completion and audit writes have no shared transaction; retries can create a second payment and reduce receivables twice.

Fix: Move posting into one transactional service and use a tenant-scoped operation key for idempotent retries.

Verify: Inject failures at each write and repeat the same operation; persist either one complete payment operation or none.

Sources: backend/src/main/java/com/stockpilot/ai/web/CustomerController.java:116
backend/src/main/java/com/stockpilot/ai/web/CustomerController.java:124
backend/src/main/java/com/stockpilot/ai/web/CustomerController.java:135

DB-04 P1 Interrupted Smart Import commits can remain stuck

Evidence: Code-confirmed; controlled crash test not run

Trigger: The process terminates after COMMITTING is saved and before success or failure status is recorded. **Impact:** A restart cannot execute the old catch block, and retries reject the persisted in-progress record indefinitely.

Fix: Add execution ownership and controlled stale-job recovery that proves no active execution exists before retrying.

Verify: Crash a disposable worker at commit boundaries, recover, and prove invoices, payments and movements are not duplicated.

Sources: backend/src/main/java/com/stockpilot/ai/service/SmartImportCommitService.java:86
backend/src/main/java/com/stockpilot/ai/service/SmartImportCommitService.java:96
backend/src/main/java/com/stockpilot/ai/service/SmartImportCommitService.java:110

DB-05 P1 Workspace reset/delete breaks after Smart Import data exists

Evidence: Code-confirmed; PostgreSQL reproduction not run

Trigger: Run a Smart Import dry run or commit, then request workspace reset or deletion. **Impact:** The deletion list omits referencing tables and deletes some parents before children, so foreign-key violations roll back the operation.

Fix: Make reset/delete cover every tenant-owned table in dependency order, including dry runs, commits/effects, adjustments, snapshots, mappings, and cashbook links.

Verify: Use isolated migrated PostgreSQL fixtures containing dry runs, committed payments, returns, and snapshots; reset/delete must succeed without touching another tenant.

Sources: backend/src/main/java/com/stockpilot/ai/service/AccountLifecycleService.java:198
backend/src/main/resources/db/migration/V11__smart_import_dry_run.sql:4
backend/src/main/resources/db/migration/V17__cashbook_manual_resolution.sql:4

DB-06 P1 Converted unit costs lose precision when persisted

Evidence: Code-confirmed schema/calculation mismatch; PostgreSQL reproduction not run

Trigger: Purchase a pack whose price divided by its base-unit count requires more than two decimal places, then sell its units. **Impact:** A one-rupee pack of 12 becomes a stored base cost of 0.08, so selling all 12 costs 0.96 while stock valuation derives the original one-rupee cost.

Fix: Use an explicit higher-precision base-unit cost representation and consistent costing across product defaults, invoice costs, and ledger values through a forward migration.

Verify: Purchase and sell complete and partial packs with repeating-decimal costs; persisted costs, stock valuation, and gross profit must reconcile without silently rewriting old records.

Sources: backend/src/main/resources/db/migration/V1__init_schema.sql:96
backend/src/main/resources/db/migration/V1__init_schema.sql:369
backend/src/main/java/com/stockpilot/ai/service/StockLedgerService.java:324

DB-07 P2 — Invoice totals and persisted line amounts can disagree after rounding

Evidence: Code-confirmed schema/calculation mismatch; PostgreSQL reproduction not run

Trigger: Create three tax-free invoice lines of quantity 0.333 at a rate of 1. **Impact:** The database stores each line total as 0.33 while the header rounds the unrounded sum 0.999 to 1.00, leaving a 0.01 reconciliation difference.

Fix: Define a single monetary rounding policy, round line amounts before aggregation, and validate request scale and bounds against database storage.

Verify: Read invoices back in a new transaction and assert line sums, header totals, taxes, discounts, and response values agree for fractional quantities and tax rates.

Sources: backend/src/main/java/com/stockpilot/ai/service/SalesService.java:106
backend/src/main/java/com/stockpilot/ai/service/SalesService.java:129
backend/src/main/resources/db/migration/V1__init_schema.sql:373

DB-08 P2 — Smart Import accepts field sizes that final business tables reject

Evidence: Code-confirmed validation/schema gap; import reproduction not run

Trigger: Import a new product with a name between 221 and 300 characters, which fits canonical staging but not the final product column. **Impact:** The row can pass matching and dry-run review but fail final commit with a database length error, rolling back otherwise valid rows.

Fix: Validate final-table length and numeric bounds during staging and dry run and report row-level errors before commit, or deliberately widen matching schema columns.

Verify: Exercise boundary and over-boundary product names, SKUs, units, HSNs, and prices; invalid data must be flagged before commit with source-row details.

Sources: backend/src/main/resources/db/migration/V10__smart_import_canonical_staging.sql:9
backend/src/main/resources/db/migration/V1__init_schema.sql:89
backend/src/main/java/com/stockpilot/ai/service/SmartImportDryRunService.java:337
backend/src/main/java/com/stockpilot/ai/service/SmartImportCommitExecutor.java:367

DATABASE AND PERSISTENCE / CONTINUED

DB-09 P2 Concurrent forecast runs can persist duplicate current results

Evidence: Code-confirmed concurrency gap; race not reproduced

Trigger: Start two forecast runs for the same tenant concurrently when no prior forecast rows exist. **Impact:** Both transactions can delete an empty set and insert a full result set because the schema has no per-product current-result uniqueness or tenant run lock.

Fix: Serialize publication per tenant or publish an identified forecast generation atomically, with a matching uniqueness guarantee.

Verify: Synchronize two concurrent runs in isolated PostgreSQL and verify only one current forecast/reorder result per intended product and warehouse survives.

Sources: backend/src/main/java/com/stockpilot/ai/service/ForecastService.java:44
backend/src/main/resources/db/migration/V1__init_schema.sql:558

DB-10 P1 Changing a product base unit reinterprets historical stock

Evidence: Code-confirmed integrity gap; API reproduction not run

Trigger: Use the product update API to change the unit of a product that already has stock movements or invoice history. **Impact:** The update replaces baseUnitId without converting stored base quantities or preserving historical conversion requirements, producing wrong units or broken profit reads.

Fix: Block base-unit changes after dependent records exist unless an explicit audited conversion workflow safely handles all historical quantities and costs.

Verify: Create stock and invoices, attempt a base-unit change, and verify it is rejected or preserves stock value, quantities, and historical dashboard calculations exactly.

Sources: backend/src/main/java/com/stockpilot/ai/service/CatalogService.java:105
backend/src/main/java/com/stockpilot/ai/service/StockLedgerService.java:272
backend/src/main/java/com/stockpilot/ai/service/DashboardService.java:198

5 FINDINGS / IM

Tally, Excel and CSV imports

Import correctness before final posting. The findings in this section were reproduced through isolated project methods; a complete browser-to-PostgreSQL rehearsal remains necessary.

IM-01 P1 Overlapping exports double-count voucher lines

Evidence: Reproduced - compiled staging method, in-memory repositories

Trigger: Upload different exports that both contain the same invoice, such as overlapping monthly and quarterly files. **Impact:** A probe combined one 10 x INR 100 invoice from two files into one voucher with two lines and a total of INR 2,000.

Fix: Preserve source-voucher boundaries and detect overlap before grouping lines or calculating document totals.

Verify: Import overlapping and non-overlapping files in either order; each invoice posts once with unchanged totals and stock effects.

Sources: backend/src/main/java/com/stockpilot/ai/service/SmartImportStagingService.java:166
backend/src/main/java/com/stockpilot/ai/service/SmartImportStagingService.java:651
backend/src/main/java/com/stockpilot/ai/service/SmartImportStagingService.java:733

IM-02 P1 Valid Excel date cells become invalid dates

Evidence: Reproduced - in-memory XLSX and actual parser methods

Trigger: An XLSX date cell uses a normal display format such as d-mmm-yy, producing 5-Oct-26. **Impact:** Both Smart Import and Classic Import parsed this valid date as null, potentially blocking otherwise valid vouchers.

Fix: Read typed Excel date cells into ISO calendar dates and explicitly support common textual export formats.

Verify: Import identical dates across common Excel formats, textual dates and leap days; preserve the intended date without silently substituting today.

Sources: backend/src/main/java/com/stockpilot/ai/service/GenericExcelImportAdapter.java:44
backend/src/main/java/com/stockpilot/ai/service/SmartImportStagingService.java:2029
backend/src/main/java/com/stockpilot/ai/service/ImportService.java:2968

IM-03 P2 CSV escaping and multiline fields are corrupted

Evidence: Reproduced - actual CSV parser in memory

Trigger: A CSV name includes an escaped inch mark or a newline inside a quoted field. **Impact:** The parser removed the quote from a 12-inch wrench name and split one quoted multiline record into two malformed rows.

Fix: Use a CSV parser that handles escaped quotes and complete logical records while preserving source row diagnostics.

Verify: Round-trip quoted commas, embedded quotes, multiline fields, blank fields and UTF-8 names without losing characters or changing record count.

Sources: backend/src/main/java/com/stockpilot/ai/service/GenericCsvImportAdapter.java:23
backend/src/main/java/com/stockpilot/ai/service/GenericCsvImportAdapter.java:57

IM-04
P1

Classic Import rejects matching party GSTINs

Evidence: Reproduced - validation method with in-memory repositories

Trigger: Multiple voucher rows refer to the same GSTIN, or that GSTIN already belongs to an existing tenant customer or supplier. **Impact:** Validation labels a legitimate match as DUPLICATE_GSTIN/ERROR, although final commit already supports party reuse.

Fix: Match consistent party identities and block only genuine conflicts in GSTIN ownership or supplied party details.

Verify: Repeated voucher lines reuse one party, existing parties remain unchanged, and conflicting party identities require review.

Sources: backend/src/main/java/com/stockpilot/ai/service/ImportService.java:2057
backend/src/main/java/com/stockpilot/ai/service/ImportService.java:2081
backend/src/main/java/com/stockpilot/ai/service/ImportService.java:2103

IM-05
P1

Custom-column rows disappear before mapping

Evidence: Reproduced - staging method with in-memory repositories

Trigger: Classic Import receives headings such as Article, Measure and Initial Qty, with none of the built-in recognized names. **Impact:** One input row produced zero staged rows; the mapping UI then has no saved row from which to recover the custom columns.

Fix: Persist all raw rows before classification, apply selected mappings, then create typed staging; report every unexplained dropped row.

Verify: Map a file using entirely custom headings and verify source, staged, validated and committed row counts reconcile.

Sources: backend/src/main/java/com/stockpilot/ai/service/ImportService.java:1821
backend/src/main/java/com/stockpilot/ai/service/ImportService.java:1865
web/app/imports/page.tsx:111

6 FINDINGS / BE

Backend, analytics and permissions

Business calculations and server-side authorization. Protect source-of-truth stock movements while fixing analytical consumption, monetary consistency and role-sensitive output.

BE-01 P1 Invoice-only sales are absent from demand analytics

Evidence: Code-confirmed; new end-to-end reproduction not run

Trigger: Smart Import posts historical sales as invoices without SALE movements to preserve a closing-stock snapshot. **Impact:** Forecast, reorder and dead-stock logic read movements, so recent sales can become zero demand and active products can appear dead.

Fix: Use shared dated invoice-item demand, normalize to base units, and reconcile movement-only history without counting invoice-backed sales twice.

Verify: Equivalent invoice-only and stock-posting histories yield the same demand; recent sales exclude a product from no-sales dead stock.

Sources: backend/src/main/java/com/stockpilot/ai/service/ForecastService.java:47
backend/src/main/java/com/stockpilot/ai/service/DeadStockActionService.java:49
backend/src/main/java/com/stockpilot/ai/service/ReorderEngineService.java:145

BE-02 P1 AI margins mix source quantities with base-unit costs

Evidence: Code-confirmed calculation mismatch

Trigger: A sale uses a box/pack unit while its recorded cost is per base piece. **Impact:** One box of 12 sold for INR 180 at INR 10 per piece yields INR 170 in best-margin analysis instead of INR 60.

Fix: Use one base-unit costing calculation throughout margin ranking and normalize purchase-cost comparisons before averaging.

Verify: Dashboard and AI margin outputs agree for piece, box, fractional-conversion and mixed-unit fixtures.

Sources: backend/src/main/java/com/stockpilot/ai/service/ProfitInsightService.java:101
backend/src/main/java/com/stockpilot/ai/service/ProfitInsightService.java:154
backend/src/main/java/com/stockpilot/ai/service/DashboardService.java:198

BE-03 P1 Restricted roles receive sensitive cost and profit fields

Evidence: Code-confirmed permissions/response mismatch

Trigger: Warehouse or other restricted staff call permitted product, stock or dashboard endpoints. **Impact:** Responses include purchase costs, valuations or profits even when the role lacks the corresponding sensitive-field permissions.

Fix: Enforce cost/profit permissions in backend response construction and check assistant evidence and report endpoints too.

Verify: Run direct API role-matrix tests: permitted roles see fields; restricted roles cannot retrieve them; tenant isolation remains intact.

Sources: backend/src/main/java/com/stockpilot/ai/service/CatalogService.java:176
backend/src/main/java/com/stockpilot/ai/service/StockLedgerService.java:79
backend/src/main/java/com/stockpilot/ai/web/DashboardController.java:24

BE-04 P1 Reorder demand uses incorrect warehouse and time scopes

Evidence: Code-confirmed query/calculation mismatch

Trigger: A product has pending orders for one branch, or sales occurred within the latest 24 hours. **Impact:** Tenant-wide pending orders affect every warehouse, and the demand window excludes recent sales, distorting branch replenishment.

Fix: Allocate pending quantities to the relevant warehouse and use explicit calendar-day demand windows that include intended current-day activity.

Verify: A branch A purchase order cannot suppress branch B replenishment; a new sale changes the expected demand input.

Sources: backend/src/main/java/com/stockpilot/ai/service/ReorderEngineService.java:62
backend/src/main/java/com/stockpilot/ai/service/ReorderEngineService.java:138
backend/src/main/java/com/stockpilot/ai/repo/Repositories.java:241

BE-05 P2 Existing stockouts are shown as not projected

Evidence: Code-confirmed calculation/UI mismatch

Trigger: A product has established sales demand but current stock reaches zero or goes negative. **Impact:** The stockout date is null; the UI says stockout is not projected and the assistant may omit the most urgent product.

Fix: Represent already-stocked-out products explicitly and distinguish them from products with insufficient demand history.

Verify: Zero and negative stock with demand appear as urgent existing stockouts in forecasts, reorder and assistant responses.

Sources: backend/src/main/java/com/stockpilot/ai/service/ForecastService.java:74
backend/src/main/java/com/stockpilot/ai/service/ReorderEngineService.java:71
backend/src/main/java/com/stockpilot/ai/service/AiAssistantService.java:171

BE-06 P1 A partial payment closes the entire reminder

Evidence: Code-confirmed business-rule mismatch

Trigger: Record a payment of INR 100 against a reminder for INR 1,000. **Impact:** The reminder is marked PAID immediately even though INR 900 remains due, hiding a necessary collection follow-up.

Fix: Track the remaining reminder balance and separate partial-payment and fully-paid states within atomic payment posting.

Verify: Partial payment preserves the remaining follow-up; cumulative full payment closes it once, including safe retry behavior.

Sources: backend/src/main/java/com/stockpilot/ai/web/CustomerController.java:125
backend/src/main/java/com/stockpilot/ai/web/CustomerController.java:128

5 FINDINGS / MB

Mobile workflows

Included for completeness. These findings are relevant if staff workflows are demonstrated later; mobile typecheck alone does not verify device or network behavior.

MB-01 P1 Repeated taps create repeated warehouse transfers

Evidence: Code-confirmed; device/network reproduction not run

Trigger: Tap Record transfer twice while the first request is pending and enough stock remains for both requests. **Impact:** The button has no pending guard and the server creates a new transfer reference for each submission.

Fix: Add an immediate submit guard and disabled/loading state, backed by an idempotency key for retries.

Verify: Rapid taps and a lost-response retry produce one transfer pair and one inventory effect.

Sources: mobile/app/transfer.tsx:27
mobile/app/transfer.tsx:63
backend/src/main/java/com/stockpilot/ai/service/StockLedgerService.java:251

MB-02 P2 Failures and expired sessions leave misleading screens

Evidence: Code-confirmed; device walkthrough not run

Trigger: Dashboard/list loading fails, or an API request returns 401 after the access token expires. **Impact:** Some promises reject without a user-visible error; missing dashboard values become zero, and token removal does not return users to login.

Fix: Add loading/error/retry states and a central authentication boundary that clears cached state and redirects on session expiry.

Verify: Offline and expired-session scenarios show actionable messages, no false zeros, and a working return to sign-in.

Sources: mobile/app/dashboard.tsx:12
mobile/app/dashboard.tsx:48
mobile/src/lib/api.ts:34

MB-03 P2 Stock adjustments show stale, cross-warehouse availability

Evidence: Code-confirmed API/UI mismatch

Trigger: Change the selected warehouse or save an adjustment on the Stock In/Out screen. **Impact:** The stock label is tenant-wide and never refreshed after saving, so it can show stock held elsewhere or an old quantity.

Fix: Fetch selected-warehouse availability and refresh it after each movement and when the screen gains focus.

Verify: Two warehouses with different balances show their own stock; a completed movement updates the visible quantity immediately.

Sources: mobile/src/components/StockAdjustmentScreen.tsx:19
mobile/src/components/StockAdjustmentScreen.tsx:51
backend/src/main/java/com/stockpilot/ai/service/CatalogService.java:164

MB-04 P2 Large catalogs are truncated in mobile operations

Evidence: Code-confirmed pagination omission

Trigger: The tenant has more than 200 products after a Tally or Excel import. **Impact:** Adjustment and transfer selectors keep only the first response page and provide no search or pagination for the remaining products.

Fix: Provide API-backed search, barcode selection and paging in stock-operation selectors.

Verify: Find and operate on products beyond position 200 without changing the server page-size limit.

Sources: mobile/src/components/StockAdjustmentScreen.tsx:19
mobile/app/transfer.tsx:16

MB-05 P2 Reorder navigation and forecast action are incomplete

Evidence: Code-confirmed navigation/action mismatch

Trigger: Try to open reorder from normal mobile navigation, or press Run forecast when suggestions already exist.
Impact: No navigation entry links the screen, and the button runs a forecast only if the current suggestion list is empty.

Fix: Add a visible reorder entry and separate refresh from an explicit forecast action that always recalculates.

Verify: Reach reorder from the app and verify every explicit run invokes the forecast endpoint and refreshes the results.

Sources: mobile/src/components/ui.tsx:8
mobile/app/dashboard.tsx:29
mobile/app/reorder.tsx:22

3 FINDINGS / OP

Setup and operational reliability

Environment and recovery behavior. Hosting findings describe the supplied configuration; they do not claim that the live deployment was inspected.

OP-01 P2 — The quick-start guide promises demo data that is disabled

Evidence: Code/configuration-confirmed

Trigger: Follow LOCAL_TESTING_GUIDE.md and start a fresh Docker Compose stack expecting the listed demo accounts. **Impact:** The guide says demo data seeds automatically, but Compose explicitly disables seeding; expected accounts and sample data are absent.

Fix: Make the guide and an explicit local-only demo profile agree, and document Java 21 plus the selected web/backend mode.

Verify: Start from a clean disposable environment using the written steps and complete login/import without hidden setup knowledge.

Sources: LOCAL_TESTING_GUIDE.md:13
docker-compose.yml:73
backend/pom.xml:24

OP-02 P1 — Hosted upload durability is absent from the supplied blueprint

Evidence: Configuration risk; live deployment not inspected

Trigger: Deploy the supplied Render configuration and replace or restart the service after uploading import source files. **Impact:** The app writes local /app/uploads files and the blueprint configures no persistent disk; source-file retention is not protected by this configuration.

Fix: Use durable object storage or a supported persistent disk and verify retention on the actual deployment.

Verify: Upload a file, redeploy/restart, then download and rebuild its import with the same content hash.

Sources: render.yaml:47
backend/src/main/java/com/stockpilot/ai/service/LocalObjectStorageService.java:21
https://render.com/docs/disks

OP-03 P2 — API requests have no application-defined deadline

Evidence: Code-confirmed reliability gap; stalled-network test not run

Trigger: The API accepts a connection but stalls during a normal request or a long-running import operation. **Impact:** The web/mobile client sets no timeout, so pending states depend on browser/OS network behavior and users have no bounded recovery path.

Fix: Set operation-appropriate deadlines; after uncertain writes, query operation status before offering any retry.

Verify: Simulate a stalled read and a lost commit response; the UI recovers clearly and never reposts a write blindly.

Sources: web/lib/api.ts:132
mobile/src/lib/api.ts:31

APPROVAL AND IMPLEMENTATION SEQUENCE

Repair and rehearsal plan

Use these batches to select implementation scope. No fixed effort estimate is asserted: database migrations and recovery design need targeted reproduction first.

Batch	Work and acceptance gate
1 - Trusted import	IM-01 to IM-05, DB-01 and DB-08. Import representative Tally XML/XLSX/CSV; preserve dates, raw rows and identity. Re-import overlapping files without duplicate vouchers or stock effects.
2 - Stored data integrity	DB-02 to DB-07, DB-09 and DB-10. Verify migrations, precision, reset/delete and concurrent operations in a disposable PostgreSQL database. Use forward migrations and preserve tenant boundaries.
3 - Business intelligence	BE-01 to BE-06. Reconcile imported invoice history, forecast demand, warehouse reorder, margin outputs and partial collection balances against known expected values.
4 - Web demonstration	FE-02 to FE-12 plus OP-01/OP-03. Create consecutive transactions, show immediate stock/balance updates, use a large catalog, switch accounts/tabs and display failures clearly. Include FE-01 only if frontend-only mode will be used.
5 - Deployment or mobile	OP-02 for hosted use; MB-01 to MB-05 for mobile. Verify source-file survival after restart, repeated taps, expired sessions, offline handling and per-warehouse availability.

A successful final rehearsal

1. Start from a disposable prepared tenant and record expected inventory, invoices and balances.
2. Upload real-format sample files; review staging, dry run and commit.
3. Re-upload an overlap and verify no double posting.
4. Create a purchase and sale; confirm stock, party balances and dashboard agree immediately.
5. Check demand, dead stock and margins against hand-calculated examples.
6. Record a partial payment, then the remainder, and confirm the follow-up status.
7. Exercise a restricted role, two browser tabs, a failed request and session expiry.
8. Repeat with the actual intended deployment configuration.

Close a finding only when its verification step passes and the affected workflow is rehearsed. Keep a list of deliberately excluded capabilities for the presenter.

KEEP THE REPORT ACCURATE

Limits and retained safeguards

Existing protections confirmed in the source

This review found the earlier customer/supplier outstanding consolidation, password-reset token revocation, basic source/base-unit valuation repairs, web session-expiry handling and CSV formula neutralization already present. They are not listed as wholly unfixed. The remaining unit-cost and cross-tab findings describe separate uncovered paths.

Intentional feature boundaries

Invoice OCR is unavailable. The current extraction service and mobile upload page explicitly report that limit. Presenting it as working OCR would be inaccurate; it is a scope limitation, not an unexpected extraction failure.

The assistant is local/rule-based. Optional external AI configuration does not itself establish a complete external-provider workflow. Describe demonstrated capabilities accurately.

Mobile preview builds use demo mode. The supplied preview build configuration forces frontend demo data. A later real-backend mobile demonstration needs a separately configured build and reachable API URL.

Other validation gaps

No complete responsive/accessibility browser review, penetration test, backup/restore drill, large-import load test, live email/Google configuration check or production monitoring verification was performed. Absence of a finding in those areas is not a pass.

No new demonstrated server-side cross-tenant read bypass was found in the inspected paths. FE-02 is a frontend identity/context synchronization problem: requests use the newly selected identity while an older tab can still display the previous context. BE-03 is sensitive-field exposure within an authorized tenant.

Report maintenance

The original source revision was clean. Application code, migrations and business data were left unchanged; only this report was produced. Revalidate source references after implementation. Keep severity, evidence status and verification results attached to each stable finding ID.

Supporting references

Repository guidance: AGENTS.md; README.md; LOCAL_TESTING_GUIDE.md. Existing audit history was used as a starting point, then checked against current source rather than treated as proof of current defects.

Render upload durability: https://render.com/docs/disks (official documentation, accessed 06 November 2026). It documents loss of local filesystem changes on restart/redeploy without persistent storage.